Device Classes

Guide	queries to find and fix device classes, auto alignment of class type of component devices, class1, class2, identifyer_1, identifyer_2, UCS manager
Team Owner	Proc and Roll
Status	active
Last Modified	`May 3, 2022`

- Introduction
- Diagnosis and Mitigation
  - gathering information about device classes from a customer system
  - what is the symbolic name of the device class of device id X
  - what device class does component discovery app id X assign to components it discovers
  - what device class should be assigned to component_did X by the app that discovered it
  - which component devices have had their class altered from the ones originally assigned by the component discovery app
  - how to fix component devices with undefined device classes: set class_type to 0 to allow discovery to reset it
  - How can you tell which device class a given dynamic app assigns from the PP source files in bitbucket
- Theory
  - table:CDB:master.definitions_dev_classes
  - table:CDB:master.dynamic_app
  - table:CDB:master_dev.component_dev_map
  - component devices and auto device class assignment
  - powerpacks create device classes
- Post Mortem Articles
  - 2024-06-27: EM-65606: Device Class Page Crashing in AP2 UI
  - 2024-06-04 EM-65277: UCS Manager PowerPack is not discovering all components
  - 2024-03-05 EM-63609: UCS Manager discovered with incorrect device class in 12.1.1
  - 2022-06-20 EM-49610: Devices Lost "Device Class" after upgrade

Introduction

Diagnosis and Mitigation

gathering information about device classes from a customer system

This script:

https://code.eng.sciencelogic.com/projects/TOOL/repos/em7_diag/browse/dump_cdb_tables.sh

dumps a number of relevant tables. This troubleshooting guide section describes the procedure for running it on a customer system: Rows-Behind Troubleshooting Guide | Rows BehindTroubleshootingGuide GatheringTableDumps:dump_cdb_tables.sh

This post mortem article has examples of analysis of device classes based on the data gathered by dump_cdb_tables: CDW - Platform 1: devices lost device class after upgrade

what is the symbolic name of the device class of device id X

```
select ld.id, ld.device, ddc.id, ddc.class
from master_dev.legend_device as ld
left join master.definitions_dev_classes as ddc on (ld.class_type=ddc.id);
where ld.id=X;
```

what device class does component discovery app id X assign to components it discovers

```
select class_type from master.dynamic_app
where aid=X;
```

what device class should be assigned to component_did X by the app that discovered it

```
select cmap.component_did, da.class_type, ddc.class, ddc.descript
from master.dynamic_app as da
join master_dev.component_dev_map as cmap on (cmap.discovered_by_aid=da.aid)
left join master.definitions_dev_classes as ddc on (ddc.id=da.class_type)
where cmap.component_did=X;
```

which component devices have had their class altered from the ones originally assigned by the component discovery app

```
select cmap.component_did, ld.class_type, ddc.class, ddc.descript, da.class_type,
ddc2.class, ddc2.descript
from master.dynamic_app as da
join master_dev.component_dev_map as cmap on (cmap.discovered_by_aid=da.aid)
join master_dev.legend_device as ld on (ld.id=cmap.component_did)
left join master.definitions_dev_classes as ddc on (ddc.id=ld.class_type)
left join master.definitions_dev_classes as ddc2 on (ddc2.id=da.class_type)
where
   ld.class_type!=da.class_type or
   ld.class_type is null or
   da.class_type is null

union

select cmap.component_did, ldv.class_type, ddc.class, ddc.descript, da.class_type,
ddc2.class, ddc2.descript
from master.dynamic_app as da
join master_dev.component_dev_map as cmap on (cmap.discovered_by_aid=da.aid)
join master_dev.legend_device_vanished as ldv on (ldv.id=cmap.component_did)
left join master.definitions_dev_classes as ddc on (ddc.id=ldv.class_type)
left join master.definitions_dev_classes as ddc2 on (ddc2.id=da.class_type)
where
   ldv.class_type!=da.class_type or
```

```
   ldv.class_type is null or
   da.class_type is null

order by component_did;
```

This converts to the following command line:

```
silo_mysql -e "select cmap.component_did, ld.class_type, ddc.class, ddc.descript, da.class_type, ddc2.class, ddc2.descript from master.dynamic_app as da join master_dev.component_dev_map as cmap on (cmap.discovered_by_aid=da.aid) join master_dev.legend_device as ld on (ld.id=cmap.component_did) left join master.definitions_dev_classes as ddc on (ddc.id=ld.class_type) left join master.definitions_dev_classes as ddc2 on (ddc2.id=da.class_type) where ld.class_type!=da.class_type or ld.class_type is null or da.class_type is null union select cmap.component_did, ldv.class_type, ddc.class, ddc.descript, da.class_type, ddc2.class, ddc2.descript from master.dynamic_app as da join master_dev.component_dev_map as cmap on (cmap.discovered_by_aid=da.aid) join master_dev.legend_device_vanished as ldv on (ldv.id=cmap.component_did) left join master.definitions_dev_classes as ddc on (ddc.id=ldv.class_type) left join master.definitions_dev_classes as ddc2 on (ddc2.id=da.class_type) where ldv.class_type!=da.class_type or ldv.class_type is null or da.class_type is null order by component_did" > report_device_classes.txt
```

how to fix component devices with undefined device classes: set class_type to 0 to allow discovery to reset it

When a device (with DID X) has a non-zero class_type that doesn't exist in master.definitions_dev_classes, it is possible to fix it with a query like this

```
update master_dev.legend_device set class_type=0 where id=X;
```

When the class_type is non-zero the backend will not attempt to alter it. A zero value will invoke logic (described in the theory section below) to determine the correct device class.

How can you tell which device class a given dynamic app assigns from the PP source files in bitbucket

This can be useful when determining if the entries in a customer's device class definitions are correct.

On my test stack, I determined that

```
silo_mysql -e "select * from master.dynamic_app" > dapp.txt
silo_mysql -e "select * from master.definitions_dev_classes" > dclasses.txt

head -1 dapp.txt  | cut -f1,2,3,4,19,20
aid     app_guid        ppguid  name    comp_dev        class_type

cat dapp.txt  | cut -f1,2,3,4,19,20 | grep -i vmware | grep -i datacenter
142     E9ABB505492337A462E4F3017C719598        0B01D6E38C4EB3E11BA0EBC67B22A23E
VMware: RootFolder Datacenter Discovery 1       338

head -1 dclasses.txt | cut -f1,2,3,4,5
id      devtype_guid    ppguid  class   descript

```

```
 cat dclasses.txt | cut -f1,2,3,4,5 | grep ^338$'\t'
338     211468AB6FB4F45A9673FC724466575B        0B01D6E38C4EB3E11BA0EBC67B22A23E
VMware  Datacenter
```

which tells us that

- the dynamic app “VMware: RootFolder Datacenter Discovery"
- assigns device class is 338
- which has devtype_guid 211468AB6FB4F45A9673FC724466575B
- class ‘VMware’
- and description ‘Datacenter’

Can we verify that this is as described by the PP source files?

If we search for the devtype_guid via Bitbucket we arrive at this file: https://code.eng.sciencelogic.com/projects/PP/repos/vmware__vsphere_base_pack_0b01d6e38c4eb3e11ba0ebc67b22a23e/browse/VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E/app/VMware__RootFolder_Datacenter_Discovery_E9ABB505492337A462E4F3017C719598/meta.json

that defines attributes for the dynamic app:


```
{
    "fields": {
        "app_guid": "E9ABB505492337A462E4F3017C719598",
        "ppguid": "0B01D6E38C4EB3E11BA0EBC67B22A23E",
        "name": "VMware: RootFolder Datacenter Discovery",
        "app_type": "15",
        "version": "8.1",
        "state": "1",
        "date_create": "1316461773",
        "poll": "5",
        "disabled_rollup": "0",
        "retention": "0",
        "norm_retention": null,
        "poll_alert": "0",
        "context": "",
        "ipp_flag": "0",
        "comp_dev": "1",
        "cache_results": "0",
        "null_row_option": "0",
        "null_col_option": "0",
        "dashboard_id": "0",
        "maxdevice_results": "50",
        "data_model": null,
        "collection_env": "5939B0BF090CC434433F200309C56294",
        "env_type": "1",
        "cu_affinity": "1"
    },
    "files": [
        "meta.descr",
        "meta.description"
```

```
    ],
    "prereq": [
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/devtype\/Datacenter_211468AB6FB4F45A9673FC724466575B"
    ],
    "child": [
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/app\/VMware__RootFolder_Datacenter_Discovery_E9ABB505492337A462E4F3017C719598\/object\/Device_Name_E6CDDD0EE709F16429E9592A98BBC236",
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/app\/VMware__RootFolder_Datacenter_Discovery_E9ABB505492337A462E4F3017C719598\/object\/Unique_Id_BF34F7EE42A6C08036FBE916F4580427",
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/app\/VMware__RootFolder_Datacenter_Discovery_E9ABB505492337A462E4F3017C719598\/soap:DatacenterDiscovery_3C66A260A60D7FE7211E7A927E6DDF30",
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/app\/VMware__RootFolder_Datacenter_Discovery_E9ABB505492337A462E4F3017C719598\/app_comp_align:01",
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/app\/VMware__RootFolder_Datacenter_Discovery_E9ABB505492337A462E4F3017C719598\/app_comp_align:160",
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/app\/VMware__RootFolder_Datacenter_Discovery_E9ABB505492337A462E4F3017C719598\/app_comp_align:161",
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/app\/VMware__RootFolder_Datacenter_Discovery_E9ABB505492337A462E4F3017C719598\/app_comp_align:162",
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/app\/VMware__RootFolder_Datacenter_Discovery_E9ABB505492337A462E4F3017C719598\/app_comp_align:163",
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/exec_env\/5939B0BF090CC434433F200309C56294"
    ]
}
```

The section that points us to the device class this app assigns is:

```
"prereq": [
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/devtype\/Datacenter_211468AB6FB4F45A9673FC724466575B"
    ],
```

We see that the devtype_guid is at the end of the path.

The attributes for that class are defined in this file:
https://code.eng.sciencelogic.com/projects/PP/repos/vmware__vsphere_base_pack_0b01d6e38c4eb3e11ba0ebc67b22a23e/browse/VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E/devtype/Datacenter_211468AB6FB4F45A9673FC724466575B/meta.json

which we quote below

```
{
    "fields": {
        "devtype_guid": "211468AB6FB4F45A9673FC724466575B",
        "ppguid": "0B01D6E38C4EB3E11BA0EBC67B22A23E",
        "class": "VMware",
        "descript": "Datacenter",
        "identifyer_1": "",
        "identifyer_2": "",
        "identifyer_3": "",
        "agent_identifier_1": "",
        "agent_identifier_2": "",
        "id2_tabular": "0",
        "weight": "1",
        "image": "vm_datacenter.png",
        "catagory": "0",
        "family_guid": "B2834837BAE26EF6821D0404B76F8B49",
        "mibed": "0",
        "virtual": "2",
        "fs_type": "0",
        "correlation_type": "0",
        "process_type": "0",
        "service_type": "0",
        "l3_topo": "0",
        "is_snmp": "0",
        "root_device": "0",
        "dashboard_id": null,
        "pdu_packing": "0",
        "intf_index_change": "0",
        "tier_id": "1"
    },
    "prereq": [
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/devcat:Virtual_Infrastructure_B2834837BAE26EF6821D0404B76F8B49",
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/sysuptime_oid:sysUptime_1.3.6.1.2.1.1.3.0"
    ],
    "child": [
"VMware__vSphere_Base_Pack_0B01D6E38C4EB3E11BA0EBC67B22A23E\/attachment\/vm_datacenter_png_2"
    ]
}
```


Theory

table:CDB:master.definitions_dev_classes

```
CREATE TABLE `definitions_dev_classes` (
  `id` int(10) unsigned NOT NULL AUTO_INCREMENT,
  `devtype_guid` char(32) DEFAULT NULL,
  `ppguid` char(32) DEFAULT NULL,
  `class` varchar(64) NOT NULL DEFAULT '',
  `descript` varchar(64) NOT NULL,
  `class_type` int(10) NOT NULL DEFAULT '0',
```

```
  `identifyer_1` varchar(255) NOT NULL DEFAULT '',
  `identifyer_2` varchar(255) NOT NULL DEFAULT '',
  `identifyer_3` varchar(255) NOT NULL,
  `agent_identifier_1` varchar(255) NOT NULL,
  `agent_identifier_2` varchar(255) NOT NULL,
  `id2_tabular` tinyint(4) NOT NULL DEFAULT '0',
  `weight` tinyint(3) unsigned NOT NULL DEFAULT '1',
  `image` varchar(32) NOT NULL DEFAULT '',
  `catagory` mediumint(8) unsigned NOT NULL DEFAULT '0',
  `family` smallint(6) unsigned NOT NULL DEFAULT '0',
  `family_guid` char(32) DEFAULT NULL,
  `mibed` tinyint(3) unsigned NOT NULL DEFAULT '0',
  `virtual` tinyint(3) unsigned NOT NULL DEFAULT '0',
  `fs_type` tinyint(3) NOT NULL DEFAULT '0',
  `correlation_type` tinyint(1) unsigned NOT NULL DEFAULT '0',
  `process_type` tinyint(3) unsigned NOT NULL DEFAULT '0',
  `service_type` tinyint(3) NOT NULL DEFAULT '0',
  `uptime_id` tinyint(4) NOT NULL DEFAULT '1',
  `l3_topo` tinyint(4) NOT NULL DEFAULT '0',
  `is_snmp` tinyint(1) NOT NULL DEFAULT '1',
  `root_device` tinyint(1) unsigned NOT NULL DEFAULT '0',
  `dashboard_id` int(11) DEFAULT NULL COMMENT 'Dashboard to be displayed for a device with this class',
  `pdu_packing` tinyint(4) NOT NULL DEFAULT '0',
  `intf_index_change` tinyint(4) NOT NULL DEFAULT '0',
  `date_edit` datetime NOT NULL DEFAULT '0000-00-00 00:00:00',
  `tier_id` mediumint(8) unsigned NOT NULL DEFAULT '0',
  PRIMARY KEY (`id`),
  UNIQUE KEY `class_type` (`class_type`),
  UNIQUE KEY `devtype_guid` (`devtype_guid`),
  KEY `class` (`class`),
  KEY `family` (`family`),
  KEY `catagory` (`catagory`),
  KEY `mibed` (`mibed`),
  KEY `virtual_id1_id3_weight` (`virtual`,`identifyer_1`,`identifyer_3`,`weight`),
  KEY `agent_identifier_1` (`virtual`,`agent_identifier_1`,`weight`),
  KEY `family_guid` (`family_guid`),
  KEY `dashboard_id` (`dashboard_id`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

Some notes on these columns

- id - this is what matches class_type in other tables.
- devtype_guid - this is the guid of the class. It is defined in the powerpack source files. When a PP is imported and installed, the device classes from that PP insert rows into this table. Because the order in which PPs are installed varies from customer to customer and release to release, the id / devtype_guid mapping from one customer system to the next varies widely.
- class_type - this is a copy of the id column. I assume it was added to make join queries against this table seem more natural.

- identifyer_1 and identifyer_2: these are matched against class1 and class2 in component_dev_map in order to automatically assign a device class to a component device

table:CDB:master.dynamic_app

```
CREATE TABLE `dynamic_app` (
  `aid` int(10) unsigned NOT NULL AUTO_INCREMENT,
  `app_guid` char(32) DEFAULT NULL,
  `ppguid` char(32) DEFAULT NULL,
  `name` varchar(64) NOT NULL,
  `app_type` tinyint(3) unsigned NOT NULL DEFAULT '0',
  `version` float NOT NULL DEFAULT '1.1',
  `state` tinyint(4) NOT NULL DEFAULT '1',
  `date_create` datetime NOT NULL DEFAULT '0000-00-00 00:00:00',
  `date_edit` datetime NOT NULL DEFAULT '0000-00-00 00:00:00',
  `user_edit` int(11) unsigned NOT NULL DEFAULT '1',
  `poll` int(10) unsigned NOT NULL DEFAULT '15',
  `disabled_rollup` tinyint(3) unsigned NOT NULL DEFAULT '0',
  `retention` mediumint(4) unsigned NOT NULL DEFAULT '90',
  `norm_retention` mediumint(4) unsigned DEFAULT NULL,
  `descr` text NOT NULL,
  `poll_alert` smallint(6) DEFAULT NULL,
  `context` varchar(128) NOT NULL,
  `ipp_flag` tinyint(4) NOT NULL DEFAULT '0',
  `comp_dev` tinyint(3) unsigned NOT NULL DEFAULT '0',
  `class_type` int(10) unsigned NOT NULL DEFAULT '0',
  `cache_results` tinyint(1) NOT NULL DEFAULT '0',
  `description` text NOT NULL,
  `null_row_option` int(10) unsigned NOT NULL DEFAULT '0',
  `null_col_option` int(10) unsigned NOT NULL DEFAULT '0',
  `dashboard_id` int(11) DEFAULT NULL COMMENT 'Dashboard to be displayed for a device with this app',
  `maxdevice_results` int(10) NOT NULL DEFAULT '50',
  `data_model` int(10) unsigned DEFAULT NULL COMMENT 'The data model an ICDA app uses',
  `collection_env` char(32) DEFAULT NULL,
  `env_type` tinyint(4) NOT NULL,
  `cu_affinity` tinyint(1) unsigned NOT NULL DEFAULT 0,
  PRIMARY KEY (`aid`),
  UNIQUE KEY `app_guid` (`app_guid`),
  KEY `poll` (`poll`),
  KEY `ppguid` (`ppguid`),
  KEY `cu_affinity` (`cu_affinity`),
  KEY `dashboard_id` (`dashboard_id`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;
```

Column notes

- class_type - the class type assigned by the dynamic app to devices discovered by it, when the class1 / class2 component identifiers do not match identifyer_1 / identifyer_2 columns of a device class definition.

table:CDB:master_dev.component_dev_map

```
CREATE TABLE `component_dev_map` (
  `id` int(10) unsigned NOT NULL AUTO_INCREMENT,
```

```
  `parent_did` int(10) unsigned NOT NULL,
  `root_did` int(10) unsigned NOT NULL,
  `component_did` int(10) unsigned NOT NULL,
  `merge_did` int(10) unsigned NOT NULL,
  `discovered_by_aid` int(10) unsigned DEFAULT NULL,
  `last_seen` datetime DEFAULT NULL,
  `unique_id` varchar(255) DEFAULT NULL,
  `dn` varchar(255) DEFAULT NULL,
  `mac` varchar(255) DEFAULT NULL,
  `class1` varchar(255) DEFAULT NULL,
  `class2` varchar(255) DEFAULT NULL,
  `uuid` varchar(255) DEFAULT NULL,
  `guid` varchar(255) DEFAULT NULL,
  `org` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`id`),
  UNIQUE KEY `component_did` (`component_did`),
  KEY `unique_id` (`root_did`,`unique_id`),
  KEY `merge_did` (`merge_did`),
  KEY `guid` (`guid`),
  KEY `parent_did` (`parent_did`),
  KEY `root_did` (`root_did`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1;

```

Column notes

- class1 and class2 are used by the backend to look for matches against master.definitions_dev_classes.{identifyer_1, identifyer_2} to auto align dynamic classes.

component devices and auto device class assignment

If you use the guide that's integrated into em7 you can find some very useful write ups. The section I quote below explains how the columns master_dev.component_dev_map.{class1, class2} are used by the backend to automatically assign a device class to a component device. To navigate to this write up, you

- open the collections tab in the dynamic app editor
- click guide
- click "component mapping and component identifiers"
- scroll down to "class identifier 1" and click the link "align a discovered component device with an existing device class"
- scroll down to "how does the sciencelogic platform use component identifiers to automatically assign a device class to each component device?"

The write up below describes fields in the GUI that map to these db columns

- master_dev.component_dev_map.{class1, class2}
- master.definitions_dev_classes.{identifyer_1, identifyer_2}

- master.dynamic_app.class_type (this is the default class type assigned when a component is discovered by the given app, but the class1 / class2 identifiers don't match the class type definitons (identifyer1/identifyer2).

Here is how that section read when I went there:

```
Using Dynamic Applications to Automatically Assign a Device Class to Component Devices

The ScienceLogic platform can automatically align a device class with each discovered component device. If you do not want to automatically
assign all discovered component devices to a single device class, you can use Component Identifiers (variables) to dynamically align component
devices with device classes. The value of two collection objects will determine the device class for each component device.

To customize the alignment between device classes and component devices, perform the following two steps.

Step 1: Create the Device Class

    Go to the Device Class Editor page (System > Customize > Device Classes).
    When you create or edit a device class for component devices, you must define these three values:

    Device Type. Select Component.
    Class Identifier 1. The manufacturer. For example, suppose you are creating the device class "Amazon Web Services: EMR Cluster". You could
    specify the value "Amazon" in this field.
    Class Identifier 2. The model. For example, suppose you are creating the device class "Amazon Web Services: EMR Cluster". You could specify
    "EMR Cluster" in this field.

NOTE: The combination of these two values should be unique to devices that subscribe to the device class.


Step 2: Define the Component Identifiers

When you create a Dynamic Application that creates component devices, you can map the value of a collection object to a component identifier. The
value collected for the collection object (from each device) will populate the component identifier (for that device).

Two of these component identifiers are used by the ScienceLogic platform to automatically align a device class with each component device. To
define these component identifiers:

    Go to the Dynamic Applications Manager page (System > Manage > Applications).
    Find the Dynamic Application that has Dynamic Component Mapping enabled. Select its wrench icon ().
    The Dynamic Applications Properties Editor page is displayed. Select the Collections tab.
    In the Collections Objects page, align a collection object with each of these component identifiers:

    Class Identifier 1. The class type of the component devices, typically the vendor. Align this component identifier with a collection object
```

that collects the value you specified in the Class Identifier 1 field for the device class.

Class Identifier 2. The class sub-type of the component devices, typically the model. Align this component identifier with a collection object that collects the value you specified in the Class Identifier 2 field for the device class.

How Does the ScienceLogic platform Use Component Identifiers to Automatically Assign a Device Class to Each Component Device?

During discovery, the ScienceLogic platform will use the following procedures to automatically assign a device class to each component device:

If the values for component identifiers Class Identifier 1 and Class Identifier 2 exactly match the values specified in the device-class fields Class Identifier 1 and Class Identifier 2 for a device class, the ScienceLogic platform will align the component device with that device class.

If the value for component identifier Class Identifier 1 exactly matches the device-class field Class Identifier 1 in a device class definition, but the component identifier Class Identifier 2 either does not match a device class description or is not defined, the ScienceLogic platform will assign the component device to the device class with the matching value in the field Class Identifier 1 and no value defined for Class Identifier 2.

NOTE: ScienceLogic recommends that you create such a device class as a "fallback" for the specified Class Identifier 1. The ScienceLogic platform can then use this device class for any component devices that have an unrecognized Class Identifier 2.

If the Dynamic Application does not include component identifiers Class Identifier 1 and Class Identifier 2 OR the values for Class Identifier 1 and Class Identifier 2 do not match the fields in an existing device class, the ScienceLogic platform aligns the component device with the device class associated with the Dynamic Application. You can define this device class in the Device Class Editor (System > Customize > Device Classes) or in the Dynamic Application Editor in the Components tab (System > Manage > Applications > edit a Dynamic Application > Components tab).

If the Dynamic Application does not include component identifiers Class Identifier 1 and Class Identifier 2 OR the values for Class Identifier 1 and Class Identifier 2 do not match the fields in an existing device class, AND the Dynamic Application is not aligned with a device class (either in the Device Class Editor page or in the Components tab), the ScienceLogic platform automatically assigns the component device to the device class "Generic: Component".

Aligning a Default Device Class with a Dynamic Application

You can align a device class with a Dynamic Application. If you choose not to use the component identifiers Class Identifier 1 and Class Identifier 2 in your Dynamic Application to match a device class, the ScienceLogic platform will automatically align each newly discovered component device with the default device class that you aligned with the Dynamic Application.

There are two ways to align a default device class with a Dynamic Application:

```
    In the Device Class Editor (System > Customize > Device Classes), in the Dynamic app Alignment field.
    In the Dynamic Application, in the Components tab (System > Manage > Applications > create/edit a dynamic application > Components tab).


Defining a Default Device Class in the Device Class Editor

To define a default device class for a Dynamic Application:

    Go to the Device Class Editor page (System > Customize > Device Classes).
    Create or edit the device class you want to align with the Dynamic Application.
    In the field Dynamic App Alignment, select a Dynamic Application.

    All component devices that are created with the selected Dynamic Application will be assigned to the current device class.


Defining a Default Device Class in the Dynamic Application

    Go to the Dynamic Applications Manager page (System > Manage > Applications).
    Find the Dynamic Application that has Dynamic Component Mapping enabled. Select its wrench icon ().
    The Dynamic Applications Properties Editor page is displayed.
    Select the Component tab. The Dynamic Application Component Alignment page is displayed.

NOTE: The Component tab and the Dynamic Application Component Alignment page do not appear for Dynamic Applications that do not have component mapping
enabled in the Dynamic Applications Properties Editor page.

    In the Device Class Alignment field, select a device class.
    All component devices that are created with the current Dynamic Application will be assigned to the selected device class.


```

powerpacks create device classes

Device classes are defined in powerpacks. When a powerpack is installed, its device classes are added to master.definitions_dev_classes. Unlike many definitions tables, there is no initial set of device classes defined under this directory https://code.eng.sciencelogic.com/projects/EM/repos/em7_v7/browse/G3code/sql/datasets/init_data/init_state

I believe this file: https://code.eng.sciencelogic.com/projects/EM/repos/em7_v7/browse/G3code/gui/ap/exec/admin_device_class_management.inc

contains the code used to insert new device classes and remove old ones when a powerpack is installed or removed.

Post Mortem Articles

2024-06-27: EM-65606: Device Class Page Crashing in AP2 UI

ts_guide_to_jira_link

Link: EM-65606: Device Class Page Crashing in AP2 UI CLOSED

From the description:

```
Observed Behavior: The Devices > Device Classes page does not load

Expected Behavior: The page should load as expected.

Summary of troubleshooting steps taken so far and the current status:
The customer reports that on 3 of their production stacks, the Devices > Device Classes page crashes when you navigate to it with a "Sorry, there was an error" message. This seems to affect all users on all 3 stacks, including the
em7admin account. All of the affected stacks are on SL1 12.1.1 build 605, however, they also have a lab on the same version and build where this issue is not occurring. When checking the nextui logs on the prod1 stack, I did see
permission errors with the ACT_USER_VIEW and DEV_CLASS_VIEW hooks, but we are reproducing the issue with admin accounts, including the em7admin account so this should not be an issue. Checking the nextui logs on the other 2 stacks
we did not see the same messages either when reproducing the issue so the access hook errors appear to be red herrings. I checked the data in the master.definitions_dev_classes table but did not see any null values or any malformed
entries. Checking the developer logs did not provide any further insight either. This only affects the AP2 UI. The System > Customize > Device Classes page in the classic UI loads fine on all affected stacks. Requesting assistance
from engineering to investigate this issue further.
```

2024-06-04 EM-65277: UCS Manager PowerPack is not discovering all components

ts_guide_to_jira_link

Link: SOL-27337: UCS Manager PowerPack is not discovering all components CLOSED

From the description:

```
Observed Behavior:  Root device is not bringing in the correct Device Category
Expected Behavior:  Root device should be showing the correct Device Category

Summary of troubleshooting steps taken so far and the current status:
https://sciencelogic.slack.com/archives/C04L3KNQLGJ/p1716906864704819

==========================================================================================
We're seeing some issues with discovering new UCS Manager devices into monitoring. The device class is recognized and DAs are being aligned, however, components are not fully building out and we're seeing errors in logs.
```

The slack link is here: https://app.slack.com/client/E03R7NUFC4X/C04L3KNQLGJ

These nmap commands were suggested for diagnosis in that thread:

```
Have the customer run these 2 nmap commands, first to a working UCS manager device's IP address, and one that is not being classified properly, so we can see the difference in success and failure:
nmap -sS -p T:443,80 -A --version-all -O -oX. <IP address>
nmap -sS -p T:443,80 --version-all -O -oX. <IP address>

So you should be returning to us 4 pieces of output: 2 from a UCS Manager that the customer is referencing in their comment of 'only seeing issues on 2 of the several ....'
And 2 from a failing one, like the IP address of the last result
```

2024-03-05 EM-63609: UCS Manager discovered with incorrect device class in 12.1.1

ts_guide_to_jira_link

Link: 💔 SOL-27558: UCS Manager discovered with incorrect device class in 12.1.1 TO DO

From the description:

```
Observed Behavior: After initial discovery UCS Manager is incorrectly classified as Apache server
Expected Behavior: Device should be classified with correct device class

Summary of troubleshooting steps taken so far and the current status:
Cisco has tested UCS monitoring in SLv 12.1.1 and they noticed after the initial discovery, it is classified as Apache server instead of UCS manager. But apps are auto aligned and collection is working fine,
 Only issue with device classification.
 Tested by onaboding the same device in 11.2.1.4 stack where it is classified correctly as UCS manager
 This UCS manager discovered in 12.1.1 has been classified as a Apache server device class. We verified the docs and noticed the "UCS Cluster Information" DA was not auto aligned, Hence we manually aligned that app.
We modified the Device Class to 'Cisco UCS Manager' this stays fine. But doing manual classification is not feasible and customer want to fix.
Enabled the Discovery Detail in debug and reproduced the issue in 12.1.1 stack and captured the logs
Need ENGG help to test this behavior and share the workaround or fix.
```

2022-06-20 EM-49610: Devices Lost "Device Class" after upgrade

Link: CDW - Platform 1: devices lost device class after upgrade

ts_guide_to_jira_link

In this case, an EM7 upgrade caused device classes for component devices to take on new values that didn't even exist in master.definition_dev_classes. It is not clear how gaps like this can be created in the ID ranges for that table because uninstalling a PP does not delete the device classes the PP installed.